

Rihla Backend Implementation Spec

Rihla — Backend Implementation Spec

For: Backend developer **From:** Frontend (UI/UX prototype complete — [rihla.html](#)) **Status:** Final — ready to build against **Stack:** Python, FastAPI, PostgreSQL, JWT auth

This spec is self-contained. It covers the data models, every endpoint with exact request/response shapes, and the decisions needed to build v1 — nothing here is pending discussion. Where a choice had to be made, it's stated as a decision, not a question.

1. Core concept

One user model, no separate driver/rider role. Any user can post a ride (acting as driver) or join a ride (acting as rider) at any time. Users only see rides from others in their own **community** (building/neighborhood/campus), scoped by a community code entered once at signup.

2. Data models

User

Field	Type	Notes
id	UUID	PK
name	string	
email	string	unique
password_hash	string	bcrypt
profile_picture_url	string	nullable
about	string	nullable, short bio shown on profile
community_id	FK → Community	nullable until onboarding completes
created_at	datetime	

Community

Field	Type	Notes
id	UUID	PK
name	string	e.g. "Cedar Heights Campus"
code	string	unique, stored uppercase, matched case-insensitively
active	bool	inactive codes are rejected at verification

Ride

Field	Type	Notes
id	UUID	PK
driver_id	FK → User	the poster
community_id	FK → Community	copied from driver at creation
origin	string	
destination	string	
date	date	
time	time	
seats_total	int	fixed at creation — see Decision 2
seats_available	int	starts equal to seats_total, decrements on join
notes	string	nullable
status	enum	active, full, cancelled
created_at	datetime	

RideJoin (junction table)

Field	Type	Notes
id	UUID	PK
ride_id	FK → Ride	
rider_id	FK → User	
joined_at	datetime	

Rides-joined / rides-posted / total-rides and the leaderboard are always **computed** from this table plus Ride — never stored as counters on User. That keeps the numbers impossible to desync.

3. Decisions (previously open questions, now resolved)

Decision 1 — Ride cancellation. A driver can cancel a ride any time before its date/time via DELETE /rides/{id}. This sets status = cancelled (soft delete, row is kept). Every rider on that ride is notified by email (see Section 7). Cancelled rides never appear in GET /rides.

Decision 2 — Seats are fixed at creation. seats_total cannot be edited after posting. If a driver wants to change seat count, they cancel and repost. This avoids a whole class of race conditions with riders who already joined against the old number.

Decision 3 — No moderation system in v1. The community code is the trust boundary for v1. No block/report/flag endpoints. Flagging this as the most likely thing to get revisited post-launch, but it is explicitly out of scope now so it doesn't block shipping.

4. Auth

JWT bearer tokens. POST /auth/login and POST /auth/register both return a token; every other endpoint requires Authorization: Bearer <token>.

GET /rides, POST /rides/{id}/join, etc. all resolve the current user from the token — never trust a user_id passed in a request body.

POST /auth/register

```
// Request
{ "name": "Behruzбек Marufjonov", "email": "behruzбек@campus.edu",
  "password": "*****" }

// 201 Response
{ "token": "eyJ...", "user": { "id": "u_1", "name": "Behruzбек
Marufjonov", "community_id": null } }
```

POST /auth/login

```
// Request
{ "email": "behruzбек@campus.edu", "password": "*****" }

// 200 Response
{ "token": "eyJ...", "user": { "id": "u_1", "name": "Behruzбек
Marufjonov", "community_id": null } }
```

community_id: null tells the frontend to route to the community code screen instead of Home.

POST /community/verify

```
// Request
{ "code": "RIHLA-2026" }

// 200 Response
{ "community": { "id": "c_1", "name": "Cedar Heights Campus" } }

// 400 Response (invalid or inactive code)
{ "error": "That code doesn't match a community. Check with your
building or campus admin." }
```

On success, the backend sets user.community_id. The error message is returned verbatim-displayable — the frontend already renders it as-is.

5. Ride endpoints

GET /rides

Scoped to current_user.community_id. Returns active and full rides only, soonest first.

```
// 200 Response
[
  {
    "id": "r_101",
    "driver": { "id": "u_4", "name": "Diego Fernandez", "initials":
"DF" },
    "origin": "Maple Court",
    "destination": "Downtown Station",
    "date": "2026-08-19",
    "time": "18:40",
    "seats_total": 3,
```

```

        "seats_available": 3,
        "notes": "Leaving right after work, happy to wait 5 min.",
        "status": "active"
    }
]

```

Empty community → []. The frontend already renders the empty state off an empty array, no special flag needed.

POST /rides

```

// Request
{
  "origin": "Maple Court",
  "destination": "Downtown Station",
  "date": "2026-08-19",
  "time": "18:40",
  "seats_total": 3,
  "notes": "Leaving right after work"
}

// 201 Response – same shape as a GET /rides item

```

DELETE /rides/{id}

Driver-only (403 if requester isn't driver_id). Sets status = cancelled, emails every current rider. Returns 204.

POST /rides/{id}/join

```

// 200 Response
{ "id": "r_101", "seats_available": 2, "status": "active" }

// 409 Response – ride is full
{ "error": "This ride is full." }

// 409 Response – already joined
{ "error": "You've already joined this ride." }

// 403 Response – driver tried to join their own ride
{ "error": "You can't join your own ride." }

```

Decrement seats_available inside a DB transaction (SELECT ... FOR UPDATE or equivalent) so two riders can't both claim the last seat. When seats_available hits 0, set status = full in the same transaction.

6. Leaderboard

GET /leaderboard?month=YYYY-MM

Defaults to the current month if omitted. Community-scoped to the requesting user.

Ranked by **distinct riders taken** — i.e. count of RideJoin rows where the ride's driver_id is that user and joined_at falls in the given month. A ride with 4 riders counts as 4, not 1.

```

// 200 Response
{
  "month": "2026-08",
  "resets_on": "2026-09-01",

```

```

        "rankings": [
          { "rank": 1, "user": { "id": "u_2", "name": "Sofia Ahmadi",
"initials": "SA" }, "riders_taken": 19 },
          { "rank": 2, "user": { "id": "u_3", "name": "Amara Okafor",
"initials": "AO" }, "riders_taken": 14 }
        ]
      }

```

This is a rolling query, not a stored monthly score — no cron job needed to “reset” it. If a historical hall-of-fame view is wanted later, that’s a v2 addition, not a blocker now.

7. Profile

GET /users/me

```

// 200 Response
{
  "id": "u_1",
  "name": "Behruzбек Marufjonov",
  "email": "behruzбек@campus.edu",
  "about": "Usually driving toward downtown after 6pm.",
  "community": { "id": "c_1", "name": "Cedar Heights Campus" },
  "stats": { "rides_joined": 7, "rides_posted": 12, "total_rides":
19 }
}

```

total_rides = rides_joined + rides_posted.

PATCH /users/me

```

// Request – any subset of these fields
{ "name": "Behruzбек Marufjonov", "about": "New bio text" }

// 200 Response – full updated user object, same shape as GET
/users/me

```

email and community_id are not editable through this endpoint.

8. Notifications (needed for Decision 1)

When a ride is cancelled, email every rider on it: subject “Your ride to {destination} was cancelled”, body naming the driver, date, and time. This is the only notification required for v1 — no push notifications, no in-app inbox.

9. Explicitly out of scope for v1

- No map view / no geolocation.
 - No in-app messaging between driver and rider.
 - No payments — the app is free.
 - No moderation/reporting (Decision 3).
 - No editing of seats_total after posting (Decision 2).
-

10. Handoff note

Field names and casing above (snake_case, ISO dates, 24-hour time) are final — the frontend will be wired to match this exactly, not the other way around. Once these endpoints are live, the mock arrays in `rih1a.html` get replaced with `fetch()` calls against them.